

April 30th, 2026

UNIVERSITY
OF MIAMI



Reliable Rentals

Database Implementation | Part 3

DSC623 – Database Systems for Data Science | Spring 2026

Prof. Vanessa Aguiar

Christian Hernandez • Gabriella Escobar Cukier • Anastasiya Drandarov

Overview

This report documents the implementation of the Reliable Rentals relational database using embedded SQL with Python and SQLite, as required by Part 3 of the project. The implementation reflects all design decisions, integrity constraints, and user transactions defined in Parts 1 and 2.

The companion file `reliable_rentals_part3.py` contains the full Python implementation, organized into three sections:

- Part A. Schema creation – 5 tables, 1 trigger, 5 indexes, and all integrity constraints.
- Part B. Data insertion – 5 tuples per table, with SELECT statements showing each table's contents.
- Part C. User transactions – the 5 queries defined in Part 2 Section C, executed against the populated database.

The script is self-contained and idempotent: each run drops existing tables and recreates the database from scratch.

A. Schema Creation

The schema implements all five entities from the logical design (OUTLET, CLIENT, STAFF, VEHICLE, HIRE_AGREEMENT) along with every integrity constraint defined in Part 2 Section D.

Constraints implemented

Primary Key Constraints (D.I)

Each table declares a single PRIMARY KEY column — `outlet_number`, `client_number`, `staff_number`, `registration_number`, and `hire_number` respectively. SQLite enforces uniqueness and NOT NULL automatically on these columns.

Referential Integrity / Foreign Keys (D.II)

The pragma PRAGMA foreign_keys = ON is enabled at connection time so SQLite enforces FK constraints. All five FK relationships use ON DELETE RESTRICT to prevent orphaned references, and ON UPDATE CASCADE so PK changes propagate automatically:

- Vehicle.outlet_number → Outlet.outlet_number
- Staff.outlet_number → Outlet.outlet_number
- HireAgreement.client_number → Client.client_number
- HireAgreement.registration_number → Vehicle.registration_number
- HireAgreement.staff_number → Staff.staff_number

Alternate Key Constraints (D.III)

- Client.driving_license_number is declared UNIQUE.
- Outlet.address is declared UNIQUE — no two outlets at the same address.

Required Data (D.IV)

Mandatory columns are declared NOT NULL. This applies to every primary key, every foreign key in HIRE_AGREEMENT, and to descriptive fields that the case study identifies as required (first_name, last_name, daily_hire_rate, etc.).

Attribute Domain Constraints (D.V)

Implemented using SQL CHECK clauses:

- salary > 0, daily_hire_rate > 0, engine_size > 0, capacity > 0.
- current_mileage ≥ 0, mileage_before ≥ 0.
- mileage_after IS NULL OR mileage_after ≥ mileage_before.
- expected_end_date ≥ start_date (table-level CHECK on HIRE_AGREEMENT).
- sex IN ('M', 'F', 'O').

General Constraints (D.VI) – Trigger

The non-overlapping-rental rule from Part 2 D.VI cannot be expressed with a CHECK clause because it spans multiple rows. It is implemented as a BEFORE INSERT trigger on HIRE_AGREEMENT named prevent_vehicle_overlap. The trigger checks for any existing agreement with the same registration_number whose date range overlaps the new one and aborts the insert with an error message:

'Vehicle already rented in an overlapping period'

Performance Indexes

Indexes are created on every foreign key column to accelerate the JOIN-heavy transactions in Part C: idx_vehicle_outlet, idx_staff_outlet, idx_hire_vehicle, idx_hire_client, idx_hire_staff.

Execution result

The schema-creation block runs without errors, confirming all constraints, the trigger, and the indexes were accepted by SQLite:

```
=====
PART A. CREATE SCHEMA
=====
Schema created successfully with 5 tables, 1 trigger, and 5 indexes.
```

Figure A.1 – Output of Part A: schema creation.

B. Data Insertion

The script inserts 5 tuples into each of the five tables — 25 rows total — then runs `SELECT * FROM <table>` for each one to verify the contents. The `HIRE_AGREEMENT` data was chosen to exercise multiple business scenarios:

- Four completed rentals where `mileage_after` has been recorded (hire numbers 20, 399, 1118, 1227).
- One ongoing rental where `mileage_after` is `NULL` (hire number 635). This row is what Transaction 3 in Part C is designed to detect.
- Client 101 appears in two different agreements, allowing Transaction 2 to return multiple rows.
- Outlet 5 has a vehicle (registration 28834) that has been rented, so Transaction 4 returns a result.
- Hire 20 spans 3 days at \$45/day, giving a clean total charge of \$135 for Transaction 5.

OUTLET

```
--- OUTLET (5 rows) ---
outlet_number      address phone_number  fax_number
1      12 Prairy Ave, Miami, FL, 33137 305-305-3333 305-305-3387
2 812 Rosemry Ave, Coral Gables, FL, 33137 305-334-4455 305-334-4567
3      403 Florida St, Miami, FL, 33145 305-388-8888 305-388-8777
4      222 Port St, Homestead, FL, 33108 786-222-9976 786-222-9872
5      Sunset Dr, Miami, FL, 33130 305-305-5565 305-305-5531
```

Figure B.1 – OUTLET table after insert.

STAFF

```
--- STAFF (5 rows) ---
staff_number first_name last_name      home_address home_phone_number date_of_birth sex date_joined_company  job_title  salary  outlet_number
45 William Bergman      20 Pinetree Rd, Miami, FL 33132 305-018-3009 1993-01-25 M 2024-04-04 IT Specialist 85000.0 3
76 Margarita Diaz 3327 Morning Ave, Coral Gables, FL 33133 786-992-3345 1992-02-20 F 2018-11-18 Sales Manager 85000.0 5
112 Li Xai 1013 Lincoln Ave, Miami, FL 33134 305-888-1115 2004-05-31 M 2024-01-01 Sales Assistant 55000.0 2
231 Daniel Jackson 333 Jefferson Ave, Miami Beach, FL 33139 305-777-1222 1995-08-22 M 2021-03-01 Sales Manager 81000.0 4
237 Mary Rosas 45 Seaside Rd, Miami, FL 33134 786-837-2776 1998-07-23 F 2022-01-01 Sales Manager 80000.0 1
```

Figure B.2 – STAFF table after insert.

VEHICLE

```

--- VEHICLE (5 rows) ---
registration_number  model  make  engine_size  capacity  current_mileage  daily_hire_rate  outlet_number
22031  Corolla  Toyota    1.5         5         12000         45.0           4
11125   RAV4  Toyota    2.0         5        30000         55.0           1
11329   Fusion  Ford     1.8         5        22000         45.0           3
16645   Rogue  Nissan    2.0         5        28000         60.0           2
28834    X5    BMW      2.0         5        10000         80.0           5

```

Figure B.3 – VEHICLE table after insert.

CLIENT

```

--- CLIENT (5 rows) ---
client_number  first_name  last_name  home_address  phone_number  date_of_birth  driving_license_number
101    Allie    Rivas    430 Stonehill St, Chicago, IL, 21117  205-112-8440  1982-02-01    DL361047299
127    Ron     Kelly    33392 Sunrise St, Birmingham, AL, 35887  509-234-3111  1977-10-12    DL9277771662
281    Marianna  Prado    4490 Roosevelt Rd, Portland, OR, 12352  874-327-3323  1975-01-29    DL837718293
1398    Jim     Thompson  504 Pond Ave, Atlanta, GA, 43167  405-277-3331  1990-03-08    DL011784763
1882    Joye    Figueroa  2226 Lakeview Ave, Atlanta, GA, 43134  405-232-3422  2005-04-19    DL3892661009

```

Figure B.4 – CLIENT table after insert.

HIRE_AGREEMENT

```

--- HIRE_AGREEMENT (5 rows) ---
hire_number  start_date  expected_end_date  mileage_before  mileage_after  client_number  registration_number  staff_number
20  2026-01-03  2026-01-05        21560        21600.0         101             11329             76
399  2024-04-15  2024-04-21        18100        18150.0         1882            11329            112
635  2026-04-03  2026-04-05         8210         NaN             127             28834            237
1118  2025-12-15  2025-12-20        10350        10500.0         101             22031             76
1227  2022-02-03  2022-02-05        12000        12065.0         1398            11125            231

```

Figure B.5 – HIRE_AGREEMENT table after insert. Hire 635 has mileage_after = NULL (NaN in pandas), indicating an ongoing rental.

C. User Transactions

Each of the five transactions defined in Part 2 Section C is implemented as an SQL query, executed against the database, and the result printed. The queries use INNER JOINS through the foreign keys defined in Part A.

Transaction 1 – Vehicles at a specific outlet and their hire rate

Query: Which vehicles are currently stored at Outlet 3, and what is their daily hire rate?

Path: Vehicle filtered by outlet_number = 3.

```

--- Transaction 1 - Vehicles at Outlet 3 (1 row) ---
registration_number  make  model  daily_hire_rate
11329  Ford  Fusion         45.0

```

Figure C.1 – Result for Transaction 1.

Only vehicle 11329 (Ford Fusion, \$45/day) is based at Outlet 3, matching the data inserted in Part B.

Transaction 2 – Hire history for a specific client

Query: Show all hire agreements signed by client 101, including the vehicle rented and the hire dates.

Path: Client → HireAgreement → Vehicle.

```
--- Transaction 2 - Hire history for client 101 (2 rows) ---
hire_number client_name start_date expected_end_date registration_number make model mileage_before mileage_after
1118 Allie Rivas 2025-12-15 2025-12-20 22031 Toyota Corolla 10350 10500
20 Allie Rivas 2026-01-03 2026-01-05 11329 Ford Fusion 21560 21600
```

Figure C.2 – Result for Transaction 2.

Client 101 (Allie Rivas) has rented two different vehicles: hire 1118 (Toyota Corolla in Dec 2025) and hire 20 (Ford Fusion in Jan 2026).

Transaction 3 – Ongoing hires (not yet returned)

Query: List all ongoing hires (not yet returned), with the client's name and the outlet where the vehicle is based.

Path: HireAgreement → Client, HireAgreement → Vehicle → Outlet. Filter: mileage_after IS NULL.

```
--- Transaction 3 - Ongoing hires (not yet returned) (1 row) ---
hire_number client_name start_date expected_end_date registration_number make model outlet_number outlet_address
635 Ron Kelly 2026-04-03 2026-04-05 28834 BMW X5 5 Sunset Dr, Miami, FL, 33130
```

Figure C.3 – Result for Transaction 3.

Hire 635 is the only ongoing rental: client Ron Kelly has BMW X5 (registration 28834) from Outlet 5, scheduled to be returned on April 5, 2026.

Transaction 4 – Clients who rented vehicles from a specific outlet

Query: List all clients who have ever rented a vehicle belonging to Outlet 5, along with the vehicle they rented and the hire dates.

Path: Outlet → Vehicle → HireAgreement → Client. The traversal involves four entities, the largest in the project.

```
--- Transaction 4 - Clients who rented vehicles from Outlet 5 (1 row) ---
client_number client_name phone_number registration_number make model start_date expected_end_date mileage_before mileage_after
127 Ron Kelly 509-234-3111 28834 BMW X5 2026-04-03 2026-04-05 8210 None
```

Figure C.4 – Result for Transaction 4.

Outlet 5 only has the BMW X5, currently on hire to Ron Kelly. The query correctly identifies him as the sole client to have rented from that outlet.

Transaction 5 – Total charge for a hire agreement

Query: What is the total charge for hire agreement 20, based on the number of days and the vehicle's daily hire rate?

Path: HireAgreement → Vehicle. Calculation uses julianday() arithmetic for inclusive day count.

```
--- Transaction 5 - Total charge for hire agreement 20 (1 row) ---
hire_number registration_number make model start_date expected_end_date daily_hire_rate number_of_days total_charge distance_driven
20          11329 Ford Fusion 2026-01-03      2026-01-05          45.0           3.0         135.0           40
```

Figure C.5 – Result for Transaction 5.

Hire 20 ran from January 3 to January 5, 2026 — 3 days at \$45/day — yielding a total charge of \$135.00. The query also returns distance_driven (40 miles), demonstrating that the same join can support both billing and usage analytics.

D. GitHub Repository

All code and documentation generated for this project (Parts 1, 2, and 3) is uploaded to a single GitHub repository. The repository is organized as follows:

reliable-rentals/

- └─ README.md Project overview and instructions
- └─ part1_conceptual_design/ Part 1 report and conceptual ER diagram
- └─ part2_logical_design/ Part 2 report and logical ER diagram
- └─ part3_implementation/ reliable_rentals_part3.py and this report

Repository link:

<https://github.com/gabriellaec/DatabaseSystemsProject>

How to run the implementation script:

```
python reliable_rentals_part3.py
```

Requirements: Python 3.8+, sqlite3 (standard library), pandas. The script creates a fresh reliable_rentals.db file in the working directory each time it runs.